# IA Aplicada com LLMs

## Aula 15 — Monitoração, telemetria e OpenTelemetry

# Roteiro

1. **Monitoração** — propósitos, sintoma × causa, caixa-preta × caixa-branca, observabilidade
2. **Telemetria** — métricas, *logs*, *traces*, e como se correlacionam
3. **O que medir** — sinais de ouro, RED, USE; percentis; cardinalidade
4. **OpenTelemetry** — o que padroniza e o que não é
5. ***Trace*, contexto e propagação** — e o caminho até o *backend*

# Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **definir** monitoração e seus propósitos, e **distingui-la** de observabilidade;
- **caracterizar** métricas, *logs* e *traces* pelo que respondem e pelo que custam;
- **escolher** o que medir com sinais de ouro, RED e USE, **usar** percentis e **estimar** cardinalidade;
- **descrever** o OpenTelemetry — API, SDK, OTLP, convenções, *Collector* — e a propagação de contexto.

# Parte 1

## Monitoração

## Monitoração: coleta contínua de dados quantitativos sobre o sistema

Coletar, processar, agregar e exibir, em tempo real, contagens, erros, tempos e tempo de vida de processos (BEYER et al., 2016).

| Propósito | Entrega |
|---|---|
| **Detectar** | que algo saiu do normal, idealmente antes do usuário |
| **Diagnosticar** | onde e desde quando — por componente, versão, intervalo |
| **Alertar** | notificação humana quando o desvio exige ação |
| **Planejar capacidade** | a tendência que diz quando a infraestrutura deixa de bastar |
| **Conferir SLOs** | a medida contínua do que foi prometido |

## SLI, SLO e o alerta por sintoma

- **SLI**: medida de um aspecto do serviço — fração de requisições abaixo de 300 ms.
- **SLO**: valor-alvo do SLI num intervalo — 99,5% abaixo de 300 ms em 28 dias.
- **Orçamento de erro**: $100\% - \mathrm{SLO}$, a falha que o serviço pode consumir.

Alerta por **sintoma** (lentidão, erro, indisponibilidade), não por **causa**
(CPU alta, disco cheio): causa sem sintoma não exige ação imediata; sintoma de causa desconhecida exige, e o alerta por causa o deixa passar.

# Caixa-preta, caixa-branca e observabilidade

```
caixa-preta    sonda externa, como um cliente     mede sintoma presente
                                                  vê o processo que caiu e não emite nada
caixa-branca   estado interno exposto pelo        mede sintoma e causa
               sistema: métricas, logs, traces    vê a degradação antes da falha
```

| | **Monitoração** | **Observabilidade** |
|---|---|---|
| Pergunta | conhecida de antemão | formulada durante a investigação |
| Forma | painel e alerta prontos | consulta exploratória sobre dados ricos |
| Falha | *known unknowns* | *unknown unknowns* |

A monitoração detecta; a observabilidade diagnostica o inédito (MAJORS; FONG-JONES; MIRANDA, 2022).

# Parte 2

## Telemetria

## Os sinais

| Sinal | O que é | Responde | Custo |
|---|---|---|---|
| **Métrica** | número agregado no tempo, com atributos — *counter*, *gauge*, *histogram* | quanto, com que tendência | baixo; perde o caso individual |
| **Log** | registro de um acontecimento, de preferência estruturado | o que houve naquele instante | proporcional ao volume |
| **Trace** | caminho de **uma** requisição, como árvore de *spans* | onde o tempo foi gasto, quem falhou | proporcional ao volume amostrado |

Complementares: **eventos** (registro estruturado e nomeado) e **perfis** (amostras de pilha: onde a CPU vai).

# Parte 3

## O que medir

## Três métodos, três recortes

| Sinais de ouro (BEYER et al., 2016) | RED — por serviço (WILKIE, 2018) | USE — por recurso (GREGG, 2013) |
|---|---|---|
| **Latência** — separar sucesso de falha | ***Rate*** — req/s | ***Utilization*** — fração do tempo ocupado |
| **Tráfego** — na unidade do sistema | ***Errors*** — falhas/s | ***Saturation*** — trabalho enfileirado |
| **Erros** — explícitos, implícitos, por política | ***Duration*** — distribuição do tempo | ***Errors*** — eventos de erro do recurso |
| **Saturação** — o recurso mais restrito | | |

RED descreve o serviço como o cliente o vê; USE, os recursos sob ele.

# Exemplo: API de pagamentos (números ilustrativos)

```
RED — POST /pagamentos, última hora
    rate ........... 420 req/s
    errors ......... 0,8 %
    duration ....... p50 85 ms   p95 210 ms   p99 1,9 s      <- cauda separada

USE — por recurso
    CPU ................ 45 %     fila de execução ~0
    pool de conexões ... 96 %     35 esperas por conexão/s    <- causa provável
    disco do banco ..... 30 %     erros 0
```

O alerta dispara sobre o p99 contra o SLO (sintoma); o *pool* entra no diagnóstico.

## A média engana; o percentil descreve uma fração

Percentil $p$: o menor $x_k$ com $k = \lceil p \cdot n/100 \rceil$ na amostra ordenada.

```
980 requisições a 80 ms  +  20 a 3 s
média = 138,4 ms     <- não descreve requisição nenhuma
p50   =  80 ms     p95 = 80 ms     p99 = 3000 ms
```

A cauda é **amplificada por *fan-out***: com $n$ dependências, cada uma lenta com probabilidade 1%,

$$P(\text{ao menos uma lenta}) = 1 - 0{,}99^{\,n} \qquad n = 100 \Rightarrow P \approx 0{,}63$$

(DEAN; BARROSO, 2013). Percentis **não se agregam**: registra-se o histograma, somam-se os *buckets*, estima-se o percentil depois.

# Cardinalidade: o custo das métricas é o número de séries

$$S \leq \prod_{i=1}^{k} |a_i| \quad (\times\, b \text{ buckets, em histograma})$$

```
método (5) × rota (40) × status (10) × 15 buckets   =            30 000 séries
+ cliente_id (200 000)                              = 6 000 000 000 séries
```

- Em **métricas**, cardinalidade é custo: atributos só de domínio pequeno.
- Em ***traces* e eventos**, é o que permite perguntas novas: `cliente_id` no *span* não cria série.
- Regra: alta cardinalidade vai no *span*; a métrica guarda o resto.

# Parte 4

## OpenTelemetry

## Origem e o que padroniza

Projeto da **CNCF**, de 2019: fusão do **OpenTracing** (API de *tracing*) com o **OpenCensus** (bibliotecas do Google). Os dois foram descontinuados.

| Componente | Papel |
|---|---|
| **API** | criar *spans*, métricas, *logs*; sem SDK, operação nula |
| **SDK** | implementação por linguagem: amostragem, lote, agregação, exportação |
| **OTLP** | protocolo de transporte, sobre gRPC ou HTTP |
| **Convenções semânticas** | nomes e significados por domínio — HTTP, banco, mensageria, GenAI |
| ***Collector*** | processo que recebe, processa e reexporta |

**Não é *backend***: não armazena, não exibe, não alerta. Termina no envio.

## *Trace*, *span* e propagação de contexto

- ***Trace***: os *spans* de uma requisição, em árvore, sob um `trace_id` de 16 *bytes*.
- ***Span***: nome, `span_id`, pai, início e fim, atributos, eventos, *status* e tipo — `SERVER`, `CLIENT`, `PRODUCER`, `CONSUMER`, `INTERNAL`.
- **Propagação**: o emissor injeta o contexto na requisição; o receptor o extrai e cria o filho.

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             versão         trace_id                span_id do pai  amostrado
```

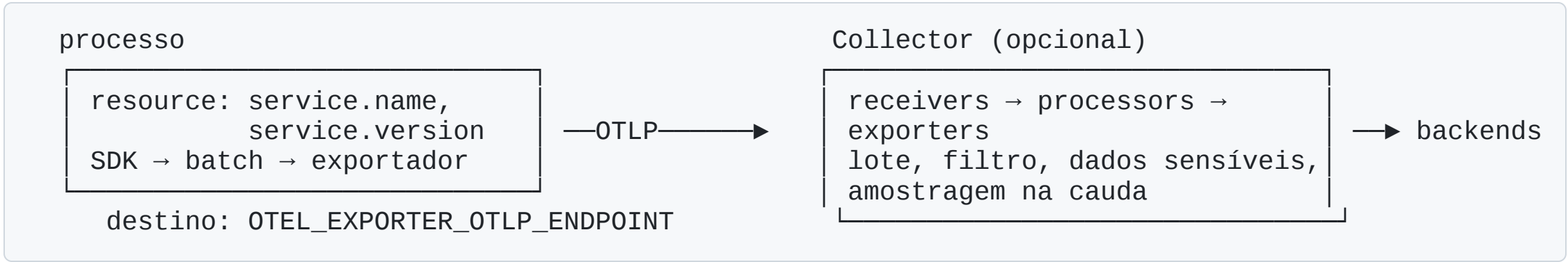
Formato **W3C Trace Context** (W3C, 2021). Sem propagação, cada serviço gera um *trace* isolado.

# Um *trace* de pagamento

```
POST /pagamentos                        SERVER  api-pagamentos          1 870 ms
├── autorizar_pagamento                 INTERNAL                        1 862 ms
│   ├── POST /antifraude/avaliar        CLIENT                             62 ms
│   │   └── POST /antifraude/avaliar SERVER  antifraude                    55 ms
│   ├── aguardar conexão do pool        INTERNAL                        1 740 ms   <- 93%
│   └── INSERT pagamentos               CLIENT  postgresql                 41 ms
```

A requisição do p99 explicada: o tempo está na espera pelo *pool*, não no antifraude nem na consulta. O *span* do antifraude é filho do de `api-pagamentos` porque o `traceparent` viajou na chamada.

# Do processo ao *backend*




- **Resource**: quem emitiu, declarado uma vez e anexado a tudo.
- **Amostragem na cabeça**: no início, por probabilidade — barata e cega ao desfecho.
- **Amostragem na cauda**: no *Collector*, após o fim — guarda erros e lentos, custa memória.

## Os *backends*: armazenam, consultam, exibem

| Plataforma | Sinal |
|---|---|
| **Prometheus** | métricas (*pull*, PromQL) |
| **Jaeger** | *traces* |
| **Grafana Tempo** / **Loki** / **Mimir** | *traces* / *logs* / métricas |
| **Grafana** | painel e alerta sobre várias fontes |

Plataformas hospedadas, como o **Grafana Cloud**, recebem OTLP diretamente. Escolher entre elas é decisão de operação, não de instrumentação: o mesmo processo exporta para qualquer uma trocando o endereço.

## Instrumentação: automática, manual e as convenções

| Automática (*zero-code*) | Manual (*code-based*) |
|---|---|
| `opentelemetry-instrument`, *agent* da JVM | chamadas à API no código da aplicação |
| cobre a fronteira: HTTP, banco, fila | cobre o domínio: *autorizar pagamento* |
| sem código, sem conhecimento de domínio | exige código, conhece o domínio |

As duas convivem: o *span* manual nasce filho do automático.

```
with tracer.start_as_current_span("autorizar_pagamento") as span:
    span.set_attribute("pagamento.metodo", pedido.metodo)
```

**Convenções semânticas** fixam os nomes: `http.request.method`, `http.response.status_code`, `http.route`, `http.server.request.duration` — e o painel RED sai sem configuração por serviço. Há convenções por domínio, inclusive